Fiche OCaml

Récursion, listes et ordre supérieur

TD Caml 1 et 2 — corrections clés, pièges et méthode 19 octobre 2026

L'idée centrale Presque tout exercice sur les listes se résout avec le même squelette : un `match` avec un cas `[]` et un cas `e :: l`. Ce qui change d'une fonction à l'autre, c'est **ce qu'on renvoie sur la liste vide** et **ce qu'on fait de `e`**.

Table des matières

1 Syntaxe de base 1

2 Les pièges à éviter 3

3 Le squelette récursif sur les listes 4

4 Corrections clés du TD 1 5

5 Corrections clés du TD 2 : l'ordre supérieur 7

6 Renvoyer un couple 8

7 Récap et méthode 9

1. Syntaxe de base

En OCaml, tout est une expression qui renvoie une valeur : un `if` comme un `match` produit un résultat.

Définir des valeurs et des fonctions

```
let x = 5                          (* une valeur *)
let succ x = x + 1                 (* une fonction *)
let succ = fun x -> x + 1          (* la même chose, écrite avec fun *)
let rec fact n =                   (* rec : la fonction s'appelle elle-même *)
  if n = 0 then 1 else n * fact (n - 1)
```

- `let` définit, `let rec` définit une fonction **récursive**. Sans `rec`, la fonction ne peut pas s'appeler.
- Pas de `rec` si la fonction ne s'appelle pas elle-même (ex. `find_power`, qui appelle `loop`).
- L'appel s'écrit **sans parenthèses ni virgules** : `f x y`, pas `f(x, y)`.
- Un argument composé se met entre parenthèses : `f (x + 1)`, `loop p f (f x)`.

Définitions locales : `let ... in`

```
let find_power x n =
  let p y = y >= n in        (* p n'existe que dans find_power *)
  let f y = y * x in
  loop p f 1
```

Une fonction définie à l'intérieur voit les paramètres de la fonction englobante (`p` utilise `n`, `f` utilise `x`).

Conditions et filtrage

```
if condition then a else b         (* les deux branches ont le même type *)

match lst with
  | [] -> ...                      (* liste vide *)
  | e :: l -> ...                  (* premier élément e, reste l *)
```

Un `if` sans `else` ne peut pas renvoyer de valeur (sauf `unit`) : il faut toujours le `else`.

Opérateurs à connaître

Opérateur	Sens	Exemple
`::`	ajoute un élément en tête	`1 :: [2; 3]` = `[1; 2; 3]`
`@`	concatène deux listes	`[1] @ [2; 3]` = `[1; 2; 3]`
`&&`	et (s'arrête si le gauche est faux)	`p e && for_all p l`
`||`	ou (s'arrête si le gauche est vrai)	`p e || exists p l`
`=` / `<>`	égalité / différence	`e = f`, `x <> 0`
`mod`	reste de la division entière	`7 mod 2` = `1`
`/`	division entière	`7 / 2` = `3`

Types

- `int`, `bool` (`true` / `false`, en minuscules), `string`, `'a list`, couples `int * string`.
- `'a` = « n'importe quel type » : `map` a le type `('a -> 'b) -> 'a list -> 'b list`.
- Une liste contient des éléments **tous du même type**, séparés par `;` : `[1; 2; 3]`.

Exceptions

Fonction	Exception levée	Quand l'utiliser
`invalid_arg "msg"`	`Invalid_argument "msg"`	l'argument lui-même est mauvais (`k` négatif)
`failwith "msg"`	`Failure "msg"`	l'argument est correct mais on ne trouve pas ce qu'on cherche

2. Les pièges à éviter

Ces erreurs sont toutes apparues pendant les TD : les relire avant un partiel évite la plupart des points perdus.

Erreur	Pourquoi ça coince	À écrire
`def f x = ...`	`def` vient de Python	`let f x = ...`
`True` / `False`	avec une majuscule, OCaml croit que c'est un constructeur	`true` / `false`
`fe`, `fx`	lu comme un seul nom de variable : « Unbound value fe »	`f e`, `(f x)`
`loop p f f x`	passe `f` puis `x` comme deux arguments séparés	`loop p f (f x)`
`if p e = true then`	pas faux, mais `p e` est déjà un booléen	`if p e then`
rappeler sur la même liste	aucun progrès : récursion infinie (stack overflow)	rappeler sur la queue `l`
oublier un cas du `match`	`Match_failure` à l'exécution	couvrir `[]`, et `[_]` si on lit deux éléments
`[_]` pour « non vide »	`[_]` = exactement un élément	`_ :: _` = au moins un
virgule entre deux motifs	fabrique un tuple, ne veut pas dire « ou »	`| ([], _) | (_, []) -> ...`
`rec` sans appel récursif	avertissement, et ça trompe le lecteur	`let` simple
nom différent de l'énoncé (`exist`)	la moulinette cherche `exists` : tous les tests échouent	le nom exact du sujet
`List.rev` interdit	refusé à la correction	ton propre `rev`

Masquer un nom dans un motif. Dans `match (l1, l2) with (e :: l1, f :: l2)`, les nouveaux `l1` et `l2` cachent les anciens : on perd la liste complète. Nomme les queues autrement : `(e :: q1, f :: q2)`.

Tester une chose et en renvoyer une autre. Dans `if p (f x) then x`, on teste la valeur suivante mais on renvoie l'actuelle : le résultat a toujours un cran de retard. Ce qu'on teste et ce qu'on renvoie doivent être la même valeur : `if p x then x`.

3. Le squelette récursif sur les listes

Une liste est soit vide `[]`, soit un premier élément collé devant le reste : `e :: l`. Toute fonction sur une liste suit donc ce modèle :

```
let rec f lst = match lst with
  | [] -> (* cas de base : ce qu'on renvoie quand il n'y a plus rien *)
  | e :: l -> (* on traite e, et on rappelle f sur le reste l *)
```

Deux questions à se poser.

1. Que renvoie-t-on sur la liste vide ? La récursion finit **toujours** par y arriver : ce cas décide du résultat pour toutes les listes.
2. Que fait-on de `e`, et comment combine-t-on avec l'appel sur `l` ?

Les motifs à connaître

Motif	Ce qu'il filtre	Exemple d'usage
`[]`	la liste vide	cas de base presque partout
`[_]` ou `[x]`	exactement un élément	`is_sorted`
`e :: l`	au moins un élément	`map`, `filter`, `exists`
`e :: f :: l`	au moins deux éléments	comparer deux voisins
`(a, b) :: l`	liste de couples, 1er couple découpé	`assoc`, `unzip`
`([], [])` sur `(l1, l2)`	deux listes vides en même temps	`identic`, `shared`
`_`	n'importe quoi (à mettre en dernier)	`| _ -> false`

Trois façons de combiner `e` avec l'appel récursif

Construire une liste (`map`, `filter`) : on met `e` (ou `f e`) devant le résultat de l'appel. L'ordre est conservé.

```
| e :: l -> f e :: map f l
```

S'arrêter dès qu'on a la réponse (`for_all`, `exists`, `find`) : on teste `e`, on renvoie tout de suite si c'est décidé, sinon on continue.

```
| e :: l -> p e || exists p l
```

Accumuler (fonction auxiliaire avec un paramètre en plus) : on empile dans un accumulateur et on le renvoie à la fin.

```
let f lst =
  let rec aux lst acc = match lst with
    | [] -> rev acc            (* empiler en tête inverse l'ordre *)
    | e :: l -> aux l (e :: acc)
```

```
  in aux lst []
```

La fonction auxiliaire sert aussi à vérifier un argument **une seule fois** avant de commencer (voir `assoc` et `gcd`).

4. Corrections clés du TD 1

Ces fonctions travaillent sur une ou deux listes, sans le module `List`.

`identic` — deux listes sont-elles identiques ?

```
let rec identic l1 l2 = match (l1, l2) with
  | ([], []) -> true
  | (e :: q1, f :: q2) -> e = f && identic q1 q2
  | _ -> false            (* une seule des deux est vide *)
```

On filtre le **couple** `(l1, l2)` pour regarder les deux listes à la fois. Le cas `_` en dernier attrape tous les cas où les longueurs diffèrent.

`shared` — éléments communs de deux listes croissantes

```
let rec shared l1 l2 = match (l1, l2) with
  | ([], _) | (_, []) -> []
  | (e :: q1, f :: q2) ->
      if e = f then e :: shared q1 q2
      else if e < f then shared q1 l2     (* on jette e seulement *)
      else shared l1 q2                   (* on jette f seulement *)
```

L'astuce vient de l'ordre croissant : on jette uniquement **le plus petit**, car il ne pourra plus jamais égaler un élément de l'autre liste. On repasse la liste **entière** (`l2` ou `l1`) pour celle qu'on n'avance pas.

Exemple : `shared [1; 3; 6; 6; 8; 9] [2; 5; 6; 6; 8; 9]` = `[6; 6; 8; 9]`.

`rev` — retourner une liste (quand `List.rev` est interdit)

```
let rev l =
  let rec aux l acc = match l with
    | [] -> acc
    | e :: q -> aux q (e :: acc)
  in aux l []
```

Empiler en tête dans un accumulateur inverse l'ordre : ici, c'est exactement ce qu'on veut.

`assoc` — valeur associée à une clé (clés triées)

```
let assoc k lst =
  if k <= 0 then invalid_arg "k not a natural"
  else
    let rec aux l = match l with
      | [] -> failwith "not found"
      | (key, value) :: q ->
          if key = k then value
          else if key > k then failwith "not found"   (* déjà trop loin *)
          else aux q
    in aux lst
```

Trois idées : la fonction auxiliaire vérifie `k` une seule fois ; le motif `(key, value) :: q` découpe le couple directement ; comme les clés sont triées, on s'arrête dès que `key > k`.

`append` et `flatten` — aplatir une liste de listes

```
let rec append l1 l2 = match l1 with
  | [] -> l2
  | e :: q -> e :: append q l2

let rec flatten l = match l with
  | [] -> []
  | sl :: q -> append sl (flatten q)
```

Exemple : `flatten [[1; 2]; [3]; [4; 5; 6]]` = `[1; 2; 3; 4; 5; 6]`. Si `@` est autorisé : `sl @ flatten q`.

`gcd` — PGCD par facteurs premiers, en une fonction

```
let gcd u v =
  if u <= 0 || v <= 0 then invalid_arg "gcd: arguments must be positive"
  else
    let rec aux u v d =
      if u = 1 || v = 1 then 1
      else if u mod d = 0 && v mod d = 0 then d * aux (u / d) (v / d) d
      else if u mod d = 0 then aux (u / d) v d
      else if v mod d = 0 then aux u (v / d) d
      else aux u v (d + 1)
    in aux u v 2
```

On décompose les deux nombres **en même temps** avec le même diviseur `d` : un facteur commun aux deux est multiplié au résultat, un facteur d'un seul côté est retiré sans compter. C'est la logique de `shared`, appliquée aux nombres.

Exemples : `gcd 45 150` = `15` ; `gcd 7 100` = `1`. Sans le test du début, `gcd 0 _` boucle à l'infini.

5. Corrections clés du TD 2 : l'ordre supérieur

Une fonction d'ordre supérieur prend une autre fonction en paramètre : elle sait **parcourir**, et la fonction reçue lui dit **quoi faire**. Un prédicat `p` est une fonction qui renvoie un booléen.

`loop` — appliquer `f` jusqu'à ce que `p` soit vrai

```
let rec loop p f x =
  if p x then x
  else loop p f (f x)
```

`p` **vérifie**, `f` **avance d'un pas**, et `loop` alterne les deux. On teste d'abord `x` lui-même, car $f^0(x) = x$.

`find_power` — première puissance de x supérieure ou égale à n

```
let find_power x n =
  let p y = y >= n in      (* on s'arrête quand y >= n *)
  let f y = y * x in       (* passer à la puissance suivante *)
  loop p f 1               (* on part de x^0 = 1 *)
```

Pas de `rec` et pas de `if` : tout le travail est dans `loop`. Version courte : `loop (fun y -> y >= n) (fun y -> y * x) 1`. Exemple : `find_power 2 10` parcourt 1, 2, 4, 8, 16 et renvoie `16`.

`map` — appliquer `f` à chaque élément

```
let rec map f lst = match lst with
  | [] -> []
  | e :: l -> f e :: map f l
```

Exemple : `map (fun x -> x * x) [1; 2; 3; 4]` = `[1; 4; 9; 16]`.

`for_all` et `exists` — tous / au moins un

```
let rec for_all p lst = match lst with
  | [] -> true                    (* vrai par défaut *)
  | e :: l -> p e && for_all p l

let rec exists p lst = match lst with
  | [] -> false
  | e :: l -> p e || exists p l
```

Ce sont deux miroirs. `&&` s'arrête dès que `p e` est faux, `||` dès que `p e` est vrai. Mettre `true` sur la liste vide de `exists` ferait répondre `true` pour **toutes** les listes.

`is_sorted` — la liste est-elle triée selon `order` ?

```
let rec is_sorted order lst = match lst with
  | [] | [_] -> true
  | e :: f :: l -> order e f && is_sorted order (f :: l)
```

`order` compare **deux** éléments, d'où le motif `e :: f :: l`. Le cas `[_]` est obligatoire : toute liste triée non vide finit par lui. On rappelle sur `f :: l` et pas sur `l`, pour comparer `f` avec son voisin suivant.

`find` — premier élément qui vérifie `p`

```
let rec find p lst = match lst with
  | [] -> failwith "not found"
  | e :: l -> if p e then e else find p l
```

`find` renvoie un **élément**, pas un booléen : quand rien ne correspond, il n'y a aucune valeur à renvoyer, donc on lève une exception.

`filter` — garder les éléments qui vérifient `p`

```
let rec filter p lst = match lst with
  | [] -> []
  | e :: l -> if p e then e :: filter p l else filter p l
```

Exemple : `filter (fun x -> x mod 2 = 0) [1; 2; 3; 4]` = `[2; 4]`.

6. Renvoyer un couple

Quand l'appel récursif renvoie un couple, on le découpe avec `let (a, b) = ... in`, puis on reconstruit un nouveau couple avec `e`.

```
| e :: l ->
    let (a, b) = f l in     (* 1. traiter d'abord le reste *)
    ...                     (* 2. placer e dans le couple *)
```

`partition` — séparer selon `p` (TD 2, ex. 2.7)

```
let rec partition p lst = match lst with
  | [] -> ([], [])
  | e :: l ->
      let (l1, l2) = partition p l in
      if p e then (e :: l1, l2)
      else (l1, e :: l2)
```

Cette version garde l'ordre sans `rev` (bonus automatique). Exemple : `partition (fun x -> x mod 2 = 0) [1; 2; 3; 4; 5]` = `([2; 4], [1; 3; 5])`.

Version avec accumulateurs : il faut retourner les deux listes à la fin.

```
let partition p lst =
  let rec aux lst l1 l2 = match lst with
    | [] -> (rev l1, rev l2)
    | e :: l ->
        if p e then aux l (e :: l1) l2
        else aux l l1 (e :: l2)
  in aux lst [] []
```

`unzip` — couper une liste de couples en deux listes

```
let rec unzip lst = match lst with
  | [] -> ([], [])
  | (a, b) :: l ->
      let (l1, l2) = unzip l in
      (a :: l1, b :: l2)
```

Pas de prédicat, donc pas de `if`. Exemple : `unzip [(1, "one"); (2, "two")]` = `([1; 2], ["one"; "two"])`.

`count` — compter les oui et les non

```
let rec count p lst = match lst with
  | [] -> (0, 0)
  | e :: l ->
      let (oui, non) = count p l in
      if p e then (oui + 1, non)
      else (oui, non + 1)
```

C'est `partition` avec des nombres : `(0, 0)` au lieu de `([], [])`, et `+ 1` au lieu de `e ::`.

7. Récap et méthode

Fonction	Renvoie	Liste vide	Ce qu'on fait de `e`
`map`	une liste	`[]`	on garde `f e` devant
`filter`	une liste	`[]`	on garde `e` si `p e`, sinon on le jette
`for_all`	`bool`	`true`	on s'arrête sur `false` si `p e` est faux
`exists`	`bool`	`false`	on s'arrête sur `true` si `p e` est vrai
`is_sorted`	`bool`	`true` (et `[_]`)	on compare `e` avec le suivant
`find`	un élément	`Failure`	on renvoie `e` si `p e`
`partition`	couple de listes	`([], [])`	`e` à gauche si `p e`, sinon à droite
`unzip`	couple de listes	`([], [])`	`a` à gauche, `b` à droite
`count`	couple d'entiers	`(0, 0)`	`+ 1` à gauche si `p e`, sinon à droite
`shared`	une liste	`[]` si une est vide	on garde si `e = f`, sinon on jette le plus petit

Méthode pour attaquer un exercice

1. **Lire le type de retour.** Une liste, un booléen, un élément ou un couple ? Ça fixe le cas de base.
2. **Chercher l'exercice qui ressemble.** La plupart des sujets sont une variante d'une fonction du tableau.
3. **Écrire le cas de base.** Que doit renvoyer la fonction sur `[]`, sachant que tous les appels y arrivent ?
4. **Écrire le cas `e :: l`.** Que fait-on de `e` ? Sur quoi rappelle-t-on ? L'appel doit avancer (sur `l`, pas sur la même liste).
5. **Vérifier les motifs.** Tous les cas sont-ils couverts ? Un motif à deux éléments demande aussi `[_]`.
6. **Dérouler à la main** sur un exemple de trois éléments : ça suffit pour voir une erreur d'ordre ou de cas de base.
7. **Ajouter des exemples d'utilisation**, dont la liste vide : l'énoncé les demande souvent.